

Zcim Project
Digital Research
CP/M® 2.0 Interface Guide

version 2025-08-07

Copyright

Copyright ©1979 by Digital Research. All rights reserved. No part of this publication may be reproduced, transmitted, transcribed, stored in a retrieval system, or translated into any language or computer language, in any form or by any means, electronic, mechanical, magnetic, optical, chemical, manual or otherwise, without the prior written permission of Digital Research, Post Office Box 579, Pacific Grove, California 93950.

<http://www.lineo.com>

DRDOS, Inc [Bryan Sparks]

Copyright © 2025 by James Burlingame.

License

The original Digital Research Inc assets license is available at

<http://cpm.z80.de/license.html>.

This *Zcim Project edition* is licensed under the Creative Commons Attribution 4.0 International license. **CC BY 4.0**.

Disclaimer

Digital Research makes no representations or warranties with respect to the contents hereof and specifically disclaims any implied warranties of merchantability or fitness for any particular purpose. Further, Digital Research reserves the right to revise this publication and to make changes from time to time in the content hereof without obligation of Digital Research to notify any person of such revision or changes.

Trademarks

CP/M is a registered trademark of Digital Research.

Printing

Zcim Project edition: version 2025-08-07

Contents

1. Introduction	1
2. Operating System Call Conventions	3
2.0.1. System Reset	10
2.0.2. Console Input	10
2.0.3. Console Output	10
2.0.4. Reader Input	11
2.0.5. Punch Output	11
2.0.6. List Output	11
2.0.7. Direct Console I/O	11
2.0.8. Get IOBYTE	12
2.0.9. Set IOBYTE	12
2.0.10. Print String	12
2.0.11. Read Console Buffer	13
2.0.12. Get Console Status	14
2.0.13. Version Number	14
2.0.14. Reset Disk System	14
2.0.15. Select Disk	15
2.0.16. Open File	15
2.0.17. Close File	16
2.0.18. Search for First	16
2.0.19. Search for Next	17
2.0.20. Delete File	17
2.0.21. Read Sequential	17
2.0.22. Write Sequential	18
2.0.23. Make File	18
2.0.24. Rename File	19
2.0.25. Return Login Vector	19
2.0.26. Return Current Disk	19
2.0.27. Set DMA Address	20
2.0.28. Get Addr(ALLOC)	20
2.0.29. Write Protect Disk	20
2.0.30. Get R/O Vector	21
2.0.31. Set File Attributes	21
2.0.32. Get Addr(Disk Parameter Block)	22
2.0.33. Set/Get User Code	22

2.0.34. Read Random	22
2.0.35. Write Random	24
2.0.36. Compute File Size	24
2.0.37. Set Random Record	25
2.0.38. Reset Drives	26
2.0.39. Access Drive	26
2.0.40. Free Drive	26
2.0.41. Write Random with Zero Fill	26
3. A Sample File-to-File Copy Program	27
4. A Sample File Dump Utility	30
5. A Sample Random Access Program	33
6. System Function Summary	39
Notes on the Zcim Project Edition	41
License	41

List of Tables

Table 1-1 CP/M Memory Organization	1
CP/M 2.0 BDOS functions	4
Table 2-2 CP/M Filetypes	6
Table 2-3 File Control Block Format	8
Table 2-4 File Control Block Fields	8
Table 2-5 Command-line Layout	9
Function 0: System Reset	10
Function 1: Console Input	10
Function 2: Console Output	10
Function 3: Reader Input	11
Function 4: Punch Output	11
Function 5: List Output	11
Function 6: Direct Console I/O	11
Function 7: Get IOBYTE	12
Function 8: Set IOBYTE	12
Function 9: Print String	12
Function 10: Read Console Buffer	13
Table 2-6 Edit Control Characters	13
Function 11: Get Console Status	14
Function 12: Version Number	14
Function 13: Reset Disk System	14

Function 14: Select Disk	15
Function 15: Open File	15
Function 16: Close File	16
Function 17: Search for First	16
Function 18: Search for Next	17
Function 19: Delete File	17
Function 20: Read Sequential	17
Function 21: Write Sequential	18
Function 22: Make File	18
Function 23: Rename File	19
Function 24: Return Login Vector	19
Function 25: Return Current Disk	19
Function 26: Set DMA Address	20
Function 27: Get Addr(ALLOC)	20
Function 28: Write Protect Disk	20
Function 29: Get R/O Vector	21
Function 30: Set File Attributes	21
Function 31: Get Addr(Disk Parameter Block)	22
Function 32: Set/Get User Code	22
Function 33: Read Random	22
Function 34: Write Random	24
Function 35: Compute File Size	24
Function 36: Set Random Record	25
Function 37: Reset Drives	26
Function 38: Access Drive	26
Function 39: Free Drive	26
Function 40: Write Random with Zero Fill	26
BDOS System Function Summary	39

1. Introduction

This chapter describes CP/M (release 2) system organization including the structure of memory and system entry points. This section provides the information you need to write programs that operate under CP/M and that use the peripheral and disk I/O facilities of the system.

CP/M is logically divided into four parts, called the Basic Input/Output System (BIOS), the Basic Disk Operating System (BDOS), the Console Command Processor (CCP), and the Transient Program Area (TPA). The BIOS is a hardware-dependent module that defines the exact low level interface with a particular computer system that is necessary for peripheral device I/O. Although a standard BIOS is supplied by Digital Research, explicit instructions are provided for field reconfiguration of the BIOS to match nearly any hardware environment, see Section 6.

The BIOS and BDOS are logically combined into a single module with a common entry point and referred to as the FDOS. The CCP is a distinct program that uses the FDOS to provide a human-oriented interface with the information that is cataloged on the back-up storage device. The TPA is an area of memory, not used by the FDOS and CCP, where various nonresident operating system commands and user programs are executed. The lower portion of memory is reserved for system information and is detailed in later sections. Memory organization of the CP/M system is shown in Table 1-1

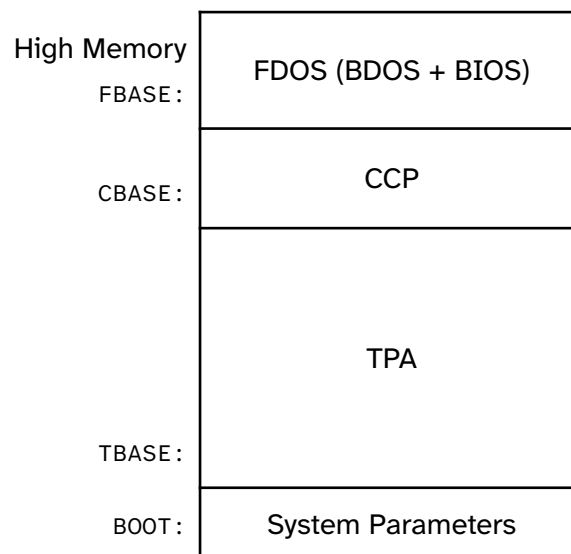


Table 1-1: CP/M Memory Organization

The exact memory addresses corresponding to `BOOT`, `TBASE`, `CBASE`, and `FBASE` vary from version to version and are described fully in Section 6.

All standard CP/M versions assume `BOOT=0000H`, which is the base of random access memory. The machine code found at location `BOOT` performs a system warm start, which loads and initializes the programs and variables necessary to return control to the CCP. Thus, transient programs need only jump to location `BOOT` to return control to CP/M at the command level.

further, the standard versions assume $TBASE=BOOT+0100H$, which is normally location $0100H$. The principal entry point to the FDOS is at location $BOOT+0005H$ (normally $0005H$) where a jump to $FBASE$ is found. The address field at $BOOT+0006H$ (normally $0006H$) contains the value of $FBASE$ and can be used to determine the size of available memory, assuming that the CCP is being overlaid by a transient program.

Transient programs are loaded into the TPA and executed as follows. The operator communicates with the CCP by typing command lines following each prompt. Each command line takes one of the following forms:

command
command file1
command file file2

where *command* is either a built-in function, such as `DIR` or `TYPE`, or the name of a transient command or program. If the *command* is a built-in function of CP/M, it is executed immediately. Otherwise, the CCP searches the currently addressed disk for a file by the name

command.COM

If the file is found, it is assumed to be a memory image of a program that executes in the TPA and thus implicitly originates at $TBASE$ in memory. The CCP loads the `COM` file from the disk into memory starting at $TBASE$ and can extend up to $CBASE$.

If the command is followed by one or two file specifications, the CCP prepares one or two File Control Block (FCB) names in the system parameter area. These optional FCBs are in the form necessary to access files through the FDOS and are described in Section 2.

The transient program receives control from the CCP and begins execution, using the I/O facilities of the FDOS. The transient program is called from the CCP. Thus, it can simply return to the CCP upon completion of its processing, or can Jump to `BOOT` to pass control back to CP/M. In the first case, the transient program must not use memory above $CBASE$, while in the latter case, memory up through $FBASE-1$ can be used.

The transient program can use the CP/M I/O facilities to communicate with the operator's console and peripheral devices, including the disk subsystem. The I/O system is accessed by passing a function number and an information address to CP/M through the FDOS entry point at $BOOT+0005H$. In the case of a disk read, for example, the transient program sends the number corresponding to a disk read, along with the address of an FCB to the CP/M FDOS. The FDOS, in turn, performs the operation and returns with either a disk read completion indication or an error number indicating that the disk read was unsuccessful.

2. Operating System Call Conventions

This section provides detailed information for performing direct operating system calls from user programs. Many of the functions listed below, however, are accessed more simply through the I/O macro library provided with the MAC macro assembler and listed in the *Zcim Project* manual entitled CP/M® MAC Macro Assembler *Language Manual and Applications Guide*.

CP/M facilities that are available for access by transient programs fall into two general categories: simple device I/O and disk file I/O. The simple device operations are:

- read a console character
- write a console character
- read a sequential character
- write a sequential character
- get or set I/O status
- print console buffer
- interrogate console ready

The following FDOS operations perform disk I/O:

- disk system reset
- drive selection
- file creation
- file close
- directory search
- file delete
- file rename
- random or sequential read
- random or sequential write
- interrogate available disks
- interrogate selected disk
- set DMA address
- set/reset file indicators.

As mentioned above, access to the FDOS functions is accomplished by passing a function number and information address through the primary point at location `BOOT+0005H`. In general, the function number is passed in register `C` with the information address in the double byte pair `DE`. Single byte values are returned in register `A`, with double byte values returned in `HL`, a zero value is returned when the function number is out of range. For reasons of compatibility, register `A = L` and register `B = H` upon return in all cases. Note that the register passing conventions of CP/M agree with those of the Intel PL/M systems programming language. CP/M functions and their numbers are listed below.

0 System Reset	19 Delete File
1 Console Input	20 Read Sequential
2 Console Output	21 Write Sequential
3 Reader Input	22 Make File
4 Punch Output	23 Rename File
5 List Output	24 Return Login Vector
6 Direct Console I/O	25 Return Current Disk
7 Get IOBYTE	26 Set DMA Address
8 Set IOBYTE	27 Get Addr(ALLOC)
9 Print String	28 Write Protect Disk
10 Read Console Buffer	29 Get R/O Vector
11 Get Console Status	30 Set File Attributes
12 Version Number	31 Get Addr(Disk Parameter Block)
13 Reset Disk System	32 Set/Get User Code
14 Select Disk	33 Read Random
15 Open File	34 Write Random
16 Close File	35 Compute File Size
17 Search for First	36 Set Random Record
18 Search for Next	

CP/M 2.0 BDOS functions

Note Function 28 and Function 32 should be avoided in application programs to maintain upward compatibility with CP/M.

Upon entry to a transient program, the CCP leaves the stack pointer set to an eight-level stack area with the CCP return address pushed onto the stack, leaving seven levels before overflow occurs. Although this stack is usually not used by a transient program (most transients return to the CCP through a jump to location 0000H) it is large enough to make CP/M system calls because the FDOS switches to a local stack at system entry. For example, the assembly-language program segment below reads characters continuously until an asterisk is encountered, at which time control returns to the CCP, assuming a standard CP/M system with BOOT=0000H.

```

BDOS    EQU 0005H    ;STANDARD CP/M ENTRY
CONIN   EQU 1        ;CONSOLE INPUT FUNCTION
;
        ORG 0100H    ;BASE OF TPA
NEXTC:  MVI C,CONIN  ;READ NEXT CHARACTER
        CALL BDOS    ;RETURN CHARACTER IN <A>
        CPI  '*'      ;END OF PROCESSING?
        JNZ  NEXTC    ;LOOP IF NOT
        RET          ;RETURN TO CCP
        END

```

CP/M implements a named file structure on each disk, providing a logical organization that allows any particular file to contain any number of records from completely empty to the full capacity of the drive. Each drive is logically distinct with a disk directory and file data area. The disk filenames are in three parts: the drive select code, the filename (consisting of one to eight non-blank characters), and the filetype (consisting of zero to three non-blank characters). The filetype names the generic category of a particular file, while the filename distinguishes individual files in each category. The filetypes listed in Table 2-2 name a few generic categories that have been established, although they are somewhat arbitrary.

Filetype	Meaning
ASM	Assembler Source
PRN	Printer Listing
HEX	Hex Machine Code
BAS	Basic Source File
INT	Intermediate Code
COM	Command File
PLI	PL/I Source File
REL	Relocatable Module
TEX	TEX Formatter Source
BAK	ED Source Backup
SYM	SID Symbol File
\$\$\$	Temporary File

Table 2-2: CP/M Filetypes

Source files are treated as a sequence of ASCII characters, where each line of the source file is followed by a carriage return, and line-feed sequence (0DH followed by 0AH). Thus, one 128-byte CP/M record can contain several lines of source text. The end of an ASCII file is denoted by a CTRL-Z character (1AH) or a real end-of-file returned by the CP/M read operation. CTRL-Z characters embedded within machine code files (for example, COM files) are ignored and the end-of-file condition returned by CP/M is used to terminate read operations.

Files in CP/M can be thought of as a sequence of up to 65536 records of 128 bytes each, numbered from 0 through 65535, thus allowing a maximum of 8 megabytes per file. Note, however, that although the records may be considered logically contiguous, they may not be physically contiguous in the disk data area. Internally, all files are divided into 16K byte segments called logical extents, so that counters are easily maintained as 8-bit values. The division into extents is discussed in the paragraphs that follow: however, they are not particularly significant for the programmer, because each extent is automatically accessed in both sequential and random access modes.

In the file operations starting with Function 15, DE usually addresses a FCB. Transient programs often use the default FCB area reserved by CP/M at location `BOOT+005CH` (normally `005CH`) for simple file operations. The basic unit of file information is a 128-byte record used for all file operations. Thus, a default location for disk I/O is provided by CP/M at location `BOOT+0080H` (normally `0080H`) which is the initial default DMA address. See Function 26.

All directory operations take place in a reserved area that does not affect write buffers as was the case in release 1, with the exception of **Search for First** and **Search for Next**, where compatibility is required.

The FCB data area consists of a sequence of 33 bytes for sequential access and a series of 36 bytes in the case when the file is accessed randomly. The default FCB, normally located at `005CH`, can be used for random access files, because the three bytes starting at `BOOT+007DH` are available for this purpose.

Field	DR	F1 ... F8	T1	T2	T3	EX	S1	S2	RC	D0 ... D15	CR	R0	R1	R2
Decimal	00	01 ... 08	09	10	11	12	13	14	15	16 ... 31	32	33	34	35
Hex	00	01 ... 08	09	0A	0B	0C	0D	0E	0F	10 ... 1F	20	21	22	23

Table 2-3: File Control Block Format

Field	Definition
DR	drive code (0-16). 0=default, 1=Drive A:, 2=B:, ... 16=P:
F1 ... F8	contain the filename in ASCII upper-case, with high bits = f1' ... f8' normally zero
T1	contains the first character of the filetype in ASCII upper-case, with high bit = t1'. Set for Read/Only file.
T2	contains the second character of the filetype in ASCII upper-case with high bit = t2'. Set for SYS file, no DIR list.
T3	contains the third character of the filetype in ASCII upper-case with high bit = t3' normally zero.
EX	contains the current extent number, normally set to 00 by the user, but in range 0-31 during file I/O
S1	reserved for internal system use
S2	reserved for internal system use. Set to zero on calls to Open , Make and Search for First
RC	record count for extent EX ; takes on values from 0-127
D0 ... D15	filled in by CP/M; reserved for system use
CR	current record to read or write in a sequential file operation; normally set to zero by user
R0	Optional Random Record Number low byte
R1	Optional Random Record Number high byte
R2	Optional Random Record Number overflow

Table 2-4: File Control Block Fields

Each file being accessed through CP/M must have a corresponding FCB, which provides the name and allocation information for all subsequent file operations. When accessing files, it is the programmer's responsibility to fill the lower 16 bytes of the FCB and initialize the **CR** field. Normally, bytes 1 through 11 are set to the ASCII character values for the filename and filetype, while all other fields are zero.

FCBs are stored in a directory area of the disk, and are brought into central memory before the programmer proceeds with file operations (see the **Open** and **Make** functions). The memory copy of the FCB is updated as file operations take place and later recorded permanently on disk at the termination of the file operation, (see the **Close** command).

The CCP constructs the first 16 bytes of two optional FCBs for a transient by scanning the remainder of the line following the transient name, denoted by *file1* and *file2* in the prototype command line described above, with unspecified fields set to ASCII blanks. The first FCB is constructed at location `BOOT+005CH` and can be used as is for subsequent file operations. The second FCB occupies the **D0 ... Dn** portion of the first FCB and must be moved to another area of memory before use. If, for example, the following command line is typed:

```
PROGNAME B:X.ZOT Y.ZAP
```

the file `PROGNAME.COM` is loaded into the TPA, and the default FCB at `BOOT+005CH` is initialized to drive code 2, filename `X`, and filetype `ZOT`. The second drive code takes the default value `0`, which is placed at `BOOT+006CH`, with the filename `Y` placed into location `BOOT+006DH` and filetype `ZAP` located 8 bytes later at `BOOT+0075H`. All remaining fields through **CR** are set to zero. Note again that it is the programmer's responsibility to move this second filename and filetype to another area, usually a separate file control block, before opening the file that begins at `BOOT+005CH`, because the open operation overwrites the second name and type.

If no filenames are specified in the original command, the fields beginning at `BOOT+005DH` and `BOOT+006DH` contain blanks. In all cases, the CCP translates lower-case alphabetic characters to upper-case to be consistent with the CP/M file naming conventions

As an added convenience, the default buffer area at location `BOOT+0080H` is initialized to the command line tail typed by the operator following the program name. The first position contains the number of characters, with the characters themselves following the character count. Given the above command line, the area beginning at `BOOT+0080H` is initialized as follows:

Offset	80	81	82	83	84	85	86	87	88	89	8A	8B	8C	8D	8E	8F	90 ... FF
Content	0EH	sp	'B'	':'	'X'	'.'	'Z'	'O'	'T'	sp	'Y'	'.'	'Z'	'A'	'P'	00H	???

Table 2-5: Command-line Layout

where the characters are translated to upper-case ASCII with uninitialized memory following the last valid character. Again, it is the responsibility of the programmer to extract the information

from this buffer before any file operations are performed, unless the default DMA address is explicitly changed.

Individual functions are described in detail in the pages that follow.

2.0.1. System Reset

Function 0: System Reset

Entry Parameters:	
Register C:	0 = 00H
Returned Value:	Does not return

The **System Reset** function returns control to the CP/M operating system at the CCP level. The CCP re-initializes the disk subsystem by selecting and logging-in disk drive A. This function has exactly the same effect as a jump to location 0000H (BOOT).

2.0.2. Console Input

Function 1: Console Input

Entry Parameters:	
Register C:	1 = 01H
Returned Value:	
Register A:	ASCII Character

The **Console Input** function reads the next console character to register A. Graphic characters, along with carriage return, line-feed, and back space (CTRL-H) are echoed to the console. Tab characters, CTRL-I, move the cursor to the next tab stop. A check is made for start/stop scroll, CTRL-S, and start/stop printer echo, CTRL-P. The FDOS does not return to the calling program until a character has been typed, thus suspending execution if a character is not ready.

2.0.3. Console Output

Function 2: Console Output

Entry Parameters:	
Register C:	2 = 02H
Register E:	ASCII character to output
Returned Value:	none

The **Console Output** function sends the ASCII character from register E to the console device. As in Function 1, tabs are expanded and checks are made for start/stop scroll and printer echo.

2.0.4. Reader Input

Function 3: Reader Input

Entry Parameters:	
Register C:	3 = 03H
Returned Value:	
Register A:	ASCII Character

The **Reader Input** function reads the next character from the logical reader into register A. Control does not return until the character has been read.

2.0.5. Punch Output

Function 4: Punch Output

Entry Parameters:	
Register C:	4 = 04H
Register E:	ASCII Character
Returned Value:	none

The **Punch Output** function sends the character from register E to the logical punch device.

2.0.6. List Output

Function 5: List Output

Entry Parameters:	
Register C:	5 = 05H
Register E:	ASCII Character
Returned Value:	none

The **List Output** function sends the ASCII character in register E to the logical listing device.

2.0.7. Direct Console I/O

Function 6: Direct Console I/O

Entry Parameters:	
Register C:	6 = 06H
Register E:	0FFH (input) or char (output)
Returned Value:	
Register A:	character or status

Direct Console I/O is supported under CP/M for those specialized applications where basic console input and output are required. Use of this function should, in general, be avoided since it bypasses all of the CP/M normal control character functions (for example, CTRL-S and CTRL-P). Programs that perform direct I/O through the BIOS under previous releases of CP/M,

however, should be changed to use direct I/O under BDOS so that they can be fully supported under future releases of MP/M and CP/M.

Upon entry to Function 6, register E either contains hexadecimal 0FFH, denoting a console input request, or an ASCII character. If the input value is 0FFH, Function 6 returns A = 00 if no character is ready, otherwise A contains the next console input character.

If the input value in E is not 0FFH, Function 6 assumes that E contains a valid ASCII character that is sent to the console.

2.0.8. Get IOBYTE

Function 7: Get IOBYTE

Entry Parameters:	
Register C:	7 = 07H
Returned Value:	
Register A:	IOBYTE value

The **Get IOBYTE** function returns the current value of IOBYTE in register A.

2.0.9. Set IOBYTE

Function 8: Set IOBYTE

Entry Parameters:	
Register C:	8 = 08H
Register E:	IOBYTE value
Returned Value:	none

The **Set IOBYTE** function changes the IOBYTE value to that given in register E.

2.0.10. Print String

Function 9: Print String

Entry Parameters:	
Register C:	9 = 09H
Registers DE:	String Address
Returned Value:	none

The **Print String** function sends the character string stored in memory at the location given by DE to the console device, until a '\$' (24H) is encountered in the string. Tabs are expanded as in Function 2, and checks are made for start/stop scroll and printer echo.

2.0.11. Read Console Buffer

Function 10: Read Console Buffer

Entry Parameters:	
Register C:	10 = 0AH
Registers DE:	Buffer Address
Returned Value:	Characters input are in the Buffer

The **Read Buffer** functions reads a line of edited console input into a buffer addressed by registers DE. Console input is terminated when either input buffer overflows or a carriage return or line-feed is typed. The Read Buffer takes the form:

DE:	+0	+1	+2	+3	+4	+5	+6	+7	+8	...	+n
	mx	nc	<i>c1</i>	<i>c2</i>	<i>c3</i>	<i>c4</i>	<i>c5</i>	<i>c6</i>	<i>c7</i>	...	??

where **m** is the maximum number of characters that the buffer will hold, 1 to 255, and **nc** is the number of characters read (set by FDOS upon return) followed by the characters read from the console. If **nc** < **mx**, then uninitialized positions follow the last character, denoted by ?? in the above figure. A number of control functions, summarized in Table 2-6, are recognized during line editing.

Table 2-6: Edit Control Characters

Character	Meaning
CTRL-C	reboots when at beginning of line
CTRL-E	causes physical end of line
CTRL-H	backspaces one character position
CTRL-J	(line-feed) terminates input line
CTRL-M	(carriage return) terminates input line
CTRL-R	retypes the current line after new line
CTRL-U	removes current line
CTRL-X	Same as CTRL-U.
rubout/del	Deletes and echoes the last character typed at the console.

The user should also note that certain functions that return the carriage to the leftmost position (for example, CTRL-X) do so only to the column position where the prompt ended. In earlier

releases, the carriage returned to the extreme left margin. This convention makes operator data input and line correction more legible.

2.0.12. Get Console Status

Function 11: Get Console Status

Entry Parameters:	
Register C:	11 = 0BH
Returned Value:	
Register A:	0FFH if a character is ready, 0 if not

The **Console Status** function checks to see if a character has been typed at the console. If a character is ready, the value 0FFH is returned in register A. Otherwise a 00H value is returned.

2.0.13. Version Number

Function 12: Version Number

Entry Parameters:	
Register C:	12 = 0CH
Returned Value:	
Registers HL:	Version Number

Function 12 provides information that allows version independent programming. A two-byte value is returned, with H = 00H designating the CP/M release (H = 01 for MP/M) and L = 00 for all releases previous to 2.0. CP/M 2.0 returns a hexadecimal 20 in register L, with subsequent version 2 releases in the hexadecimal range 21, 22, through 2F. Using Function 12, for example, the user can write application programs that provide both sequential and random access functions.

2.0.14. Reset Disk System

Function 13: Reset Disk System

Entry Parameters:	
Register C:	13 = 0DH
Returned Value:	
Register A:	0FFH if \$ file present, 0 if not

The **Reset Disk** function is used to programmatically restore the file system to a reset state where all disks are set to Read-Write. See functions 28 and 29, only disk drive A is selected, and the default DMA address is reset to 0080H. This function can be used, for example, by an application program that requires a disk change without a system reboot.

This function is normally called by the CCP after the system reset and before any other activity. If the CCP is not executed, this function must be called by what replaced it.

In 1.x and 2.x version the return value depends upon the presence of a file with a name starting with a dollar-sign (\$) on drive A. If such a file is present, a 0FFH value is returned, otherwise a zero is returned.

2.0.15. Select Disk

Function 14: Select Disk

Entry Parameters:	
Register C:	14 = 0EH
Register E:	Drive number: 0 for A, 1 for B, ...
Returned Value:	
Register A:	0 if successful, 0FFH on error

The **Select Disk** function designates the disk drive named in register E as the default disk for subsequent file operations, with E = 0 for drive A, 1 for drive B, and so on through 15, corresponding to drive P in a full 16 drive system. The drive is placed in an on-line status, which activates its directory until the next cold start, warm start, or disk system reset operation. If the disk medium is changed while it is on-line, the drive automatically goes to a Read-Only status in a standard CP/M environment, see Function 28. FCBs that specify drive code zero (**DR** = 00H) automatically reference the currently selected default drive. Drive code values between 1 and 16 ignore the selected default drive and directly reference drives A through P.

2.0.16. Open File

Function 15: Open File

Entry Parameters:	
Register C:	15 = 0FH
Registers DE:	FCB Address
Returned Value:	
Register A:	Directory Code

The **Open File** operation is used to activate a file that currently exists in the disk directory for the currently active user number. The FDOS scans the referenced disk directory for a match in positions 1 through 14 of the FCB referenced by DE (byte **S1** is automatically zeroed) where an ASCII question mark '?' (3FH) matches any directory character in any of these positions. Normally, no question marks are included, and bytes **EX** and **S2** of the FCB are zero.

If a directory element is matched, the relevant directory information is copied into bytes **D0** through **Dn** of FCB, thus allowing access to the files through subsequent read and write operations. The user should note that an existing file must not be accessed until a successful

open operation is completed. Upon return, the open function returns a directory code with the value 0 through 3 if the open was successful or 0FFH (255 decimal) if the file cannot be found. If question marks occur in the FCB, the first matching FCB is activated. Note that the current record, (**CR**) must be zeroed by the program if the file is to be accessed sequentially from the first record.

2.0.17. Close File

Function 16: Close File

Entry Parameters:	
Register C:	16 = 10H
Registers DE:	FCB Address
Returned Value:	
Register A:	Directory Code

The **Close** File function performs the inverse of the **Open** File function. Given that the FCB addressed by DE has been previously activated through an **Open** or **Make** function, the **Close** function permanently records the new FCB in the reference disk directory see functions 15 and 22. The FCB matching process for the close is identical to the open function. The directory code returned for a successful close operation is 0, 1, 2, or 3, while a 0FFH (255 decimal) is returned if the filename cannot be found in the directory. A file need not be closed if only read operations have taken place. If write operations have occurred, the close operation is necessary to record the new directory information permanently.

2.0.18. Search for First

Function 17: Search for First

Entry Parameters:	
Register C:	17 = 11H
Registers DE:	FCB Address
Returned Value:	
Register A:	Directory Code

Search for First scans the directory for a match with the file given by the FCB addressed by DE. The value 255 (hexadecimal 0FFH) is returned if the file is not found; otherwise, 0, 1, 2, or 3 is returned indicating the file is present. When the file is found, the current DMA address is filled with the record containing the directory entry, and the relative starting position is $A * 32$ (that is, rotate the A register left 5 bits, or ADD A five times). Although not normally required for application programs, the directory information can be extracted from the buffer at this position. An ASCII question mark '?' (63 decimal, 3F hexadecimal) in any position from **F1** through **EX** matches the corresponding field of any directory entry on the default or auto-selected disk drive.

If the **DR** field contains an ASCII question mark, the auto disk select function is disabled and the default disk is searched, with the search function returning any matched entry, allocated or free, belonging to any user number. This latter function is not normally used by application programs, but it allows complete flexibility to scan all current directory values. If the **DR** field is not a question mark, the **S2** byte is automatically zeroed.

2.0.19. Search for Next

Function 18: Search for Next

Entry Parameters:	
Register C:	18 = 12H
Returned Value:	
Register A:	Directory Code

The **Search for Next** function is similar to the **Search for First** function, except that the directory scan continues from the last matched entry. Similar to Function 17, Function 18 returns the decimal value 255 in A when no more directory items match.

2.0.20. Delete File

Function 19: Delete File

Entry Parameters:	
Register C:	19 = 13H
Registers DE:	FCB Address
Returned Value:	
Register A:	Directory Code

The **Delete** File function removes files that match the FCB addressed by DE. The filename and type may contain ambiguous references (that is, question marks in various positions), but the drive select code (**DR**) cannot be ambiguous, as in the **Search for First** and **Search for Next** functions.

Function 19 returns a decimal 255 if the referenced file or files cannot be found; otherwise, a value in the range 0 to 3 returned.

2.0.21. Read Sequential

Function 20: Read Sequential

Entry Parameters:	
Register C:	20 = 14H
Registers DE:	FCB Address
Returned Value:	
Register A:	Directory Code

Given that the FCB addressed by DE has been activated through an **Open** or **Make** function, the **Read Sequential** function reads the next 128-byte record from the file into memory at the current DMA address. The record is read from position **CR** of the extent, and the **CR** field is automatically incremented to the next record position. If the **CR** field overflows, the next logical extent is automatically opened and the **CR** field is reset to zero in preparation for the next read operation. The value 00H is returned in the A register if the read operation was successful, while a nonzero value is returned if no data exist at the next record position (for example, end-of-file occurs).

2.0.22. Write Sequential

Function 21: Write Sequential

Entry Parameters:	
Register C:	21 = 15H
Registers DE:	FCB Address
Returned Value:	
Register A:	Directory Code

Given that the FCB addressed by DE has been activated through an **Open** or **Make** function, the **Write Sequential** function writes the 128-byte data record at the current DMA address to the file named by the FCB. The record is placed at position **CR** of the file, and the **CR** field is automatically incremented to the next record position. If the **CR** field overflows, the next logical extent is automatically opened and the **CR** field is reset to zero in preparation for the next write operation. Write operations can take place into an existing file, in which case, newly written records overlay those that already exist in the file. Register A = 00H upon return from a successful write operation, while a nonzero value indicates an unsuccessful write caused by a full disk.

2.0.23. Make File

Function 22: Make File

Entry Parameters:	
Register C:	22 = 16H
Registers DE:	FCB Address
Returned Value:	
Register A:	Directory Code

The **Make** File operation is similar to the **Open** File operation except that the FCB must name a file that does not exist in the currently referenced disk directory (that is, the one named explicitly by a nonzero **DR** code or the default disk if **DR** is zero). The FDOS creates the file and initializes both the directory and main memory value to an empty file. The programmer must

ensure that no duplicate filenames occur, and a preceding delete operation is sufficient if there is any possibility of duplication. Upon return, register A = 0, 1, 2, or 3 if the operation was successful and 0FFH (255 decimal) if no more directory space is available. The Make function has the side effect of activating the FCB and thus a subsequent open is not necessary.

2.0.24. Rename File

Function 23: Rename File

Entry Parameters:	
Register C:	23 = 17H
Registers DE:	FCB Address
Returned Value:	
Register A:	Directory Code

The **Rename** function uses the FCB addressed by DE to change all occurrences of the file named in the first 16 bytes to the file named in the second 16 bytes. The drive code **DR** at position 0 is used to select the drive, while the drive code for the new filename at position 16 of the FCB is assumed to be zero. Upon return, register A is set to a value between 0 and 3 if the rename was successful and 0FFH (255 decimal) if the first filename could not be found in the directory scan.

2.0.25. Return Login Vector

Function 24: Return Login Vector

Entry Parameters:	
Register C:	24 = 18H
Returned Value:	
Registers HL:	Log-in Vector

The log-in vector value returned by CP/M is a 16-bit value in HL, where the least significant bit of L corresponds to the first drive A and the high-order bit of H corresponds to the sixteenth drive, labeled P. A 0 bit indicates that the drive is not on-line, while a 1 bit marks a drive that is actively on-line as a result of an explicit disk drive selection or an implicit drive select caused by a file operation that specified a nonzero **DR** field. The user should note that compatibility is maintained with earlier releases, because registers A and L contain the same values upon return.

2.0.26. Return Current Disk

Function 25: Return Current Disk

Entry Parameters:	
Register C:	25 = 19H
Returned Value:	
Register A:	Current Disk. 0 for A, ... 15 for P

Function 25 returns the currently selected default disk number in register A. The disk numbers range from 0 through 15 corresponding to drives A through P.

2.0.27. Set DMA Address

Function 26: Set DMA Address

Entry Parameters:	
Register C:	26 = 1AH
Registers DE:	DMA Address
Returned Value:	none

DMA is an acronym for Direct Memory Address, which is often used in connection with disk controllers that directly access the memory of the mainframe computer to transfer data to and from the disk subsystem. Although many computer systems use non-DMA access (that is, the data is transferred through programmed I/O operations), the DMA address has, in CP/M, come to mean the address at which the 128-byte data record resides before a disk write and after a disk read. Upon cold start, warm start, or disk system reset, the DMA address is automatically set to `BOOT+0080H`. The **Set DMA** function can be used to change this default value to address another area of memory where the data records reside. Thus, the DMA address becomes the value specified by DE until it is changed by a subsequent **Set DMA** function, cold start, warm start, or disk system reset.

2.0.28. Get Addr(ALLOC)

Function 27: Get Addr(ALLOC)

Entry Parameters:	
Register C:	27 = 1BH
Returned Value:	
Registers HL:	ALLOC Address

An allocation vector (ALLOC) is maintained in main memory for each on-line disk drive. Various system programs use the information provided by the allocation vector to determine the amount of remaining storage (see the STAT program). Function 27 returns the base address of the allocation vector for the currently selected disk drive. However, the allocation information might be invalid if the selected disk has been marked Read-Only. This function is not normally used by application programs.

2.0.29. Write Protect Disk

Function 28: Write Protect Disk

Entry Parameters:	
Register C:	28 = 1CH
Returned Value:	none

The Write Protect Disk function provides temporary write protection for the currently selected disk. Any attempt to write to the disk before the next cold or warm start operation produces the message:

BDOS Err On *d*:R/O

2.0.30. Get R/O Vector

Function 29: Get R/O Vector

Entry Parameters:	
Register C:	29 = 1DH
Returned Value:	
Registers HL:	R/O Vector Value

Function 29 returns a bit vector in register pair HL, which indicates drives that have the temporary Read-Only bit set. As in Function 24, the least significant bit corresponds to drive A, while the most significant bit corresponds to drive P. The R/O bit is set either by an explicit call to Function 28 or by the automatic software mechanisms within CP/M that detect changed disks.

2.0.31. Set File Attributes

Function 30: Set File Attributes

Entry Parameters:	
Register C:	30 = 1EH
Registers DE:	FCB Address
Returned Value:	
Register A:	Directory Code

The Set File Attributes function allows programmatic manipulation of permanent indicators attached to files. In particular, the R/O and System attributes (*t1* ' and *t2* ') can be set or reset. The DE pair addresses an unambiguous filename with the appropriate attributes set or reset. Function 30 searches for a match and changes the matched directory entry to contain the selected indicators. Indicators *f1* ' through *f4* ' are not currently used, but may be useful for applications programs, since they are not involved in the matching process during file open and close operations. Indicators *f5* ' through *f8* ' and *t3* ' are reserved for future system expansion.

2.0.32. Get Addr(Disk Parameter Block)

Function 31: Get Addr(Disk Parameter Block)

Entry Parameters:	
Register C:	31 = 1FH
Returned Value:	
Registers HL:	DPB Address

The address of the BIOS resident disk parameter block (**DPB**) is returned in HL as a result of this function call. This address can be used for either of two purposes. First, the disk parameter values can be extracted for display and space computation purposes, or transient programs can dynamically change the values of current disk parameters when the disk environment changes, if required. Normally, application programs will not require this facility.

2.0.33. Set/Get User Code

Function 32: Set/Get User Code

Entry Parameters:	
Register C:	32 = 20H
Register E:	0FFH (get) or <i>User Code</i> (set)
Returned Value:	
Register A:	Current code (get) or none (set)

An application program can change or interrogate the currently active user number by calling Function 32. If register E = 0FFH, the value of the current user number is returned in register A, where the value is in the range of 0 to 15. If register E is not 0FFH, the current user number is changed to the value of E, modulo 16.

2.0.34. Read Random

Function 33: Read Random

Entry Parameters:	
Register C:	33 = 21H
Registers DE:	FCB Address
Returned Value:	
Register A:	Return code (see details)

The **Read Random** function is similar to the sequential file read operation of previous releases, except that the read operation takes place at a particular record number, selected by the 24-bit value constructed from the 3-byte field following the FCB (byte positions **R0** at 33, **R1** at 34, and **R3** at 35). The user should note that the sequence of 24 bits is stored with least significant byte first (**R0**), middle byte next (**R1**), and high byte last (**R2**). CP/M does not reference byte **R2**,

except in computing the size of a file (Function 35). Byte **R2** must be zero, however, since a nonzero value indicates overflow past the end of file.

Thus, the **R0, R1** byte pair is treated as a double-byte, or word value, that contains the record to read. This value ranges from 0 to 65535, providing access to any particular record of the 8-megabyte file. To process a file using random access, the base extent (extent 0) must first be opened. Although the base extent might or might not contain any allocated data, this ensures that the file is properly recorded in the directory and is visible in **DIR** requests. The selected record number is then stored in the random record field (**R0, R1**), and the BDOS is called to read the record.

Upon return from the call, register A either contains an error code, as listed below, or the value 00, indicating the operation was successful. In the latter case, the current DMA address contains the randomly accessed record. Note that contrary to the sequential read operation, the record number is not advanced. Thus, subsequent random read operations continue to read the same record.

Upon each random read operation, the logical extent and current record values are automatically set. Thus, the file can be sequentially read or written, starting from the current randomly accessed position. However, note that, in this case, the last randomly read record will be reread as one switches from random mode to sequential read and the last record will be rewritten as one switches to a sequential write operation. The user can simply advance the random record position following each random read or write to obtain the effect of sequential I/O operation.

Error codes returned in register A following a random read are listed below.

- 01 reading unwritten data
- 02 (not returned in random mode)
- 03 cannot close current extent
- 04 seek to unwritten extent
- 05 (not returned in read mode)
- 06 seek past physical end of disk
- 09 invalid FCB
- 10 media changed
- 0FFH hardware error (on 3.x)

Error codes 01 and 04 occur when a random read operation accesses a data block that has not been previously written or an extent that has not been created, which are equivalent conditions. Error code 03 does not normally occur under proper system operation. If it does, it can be

cleared by simply rereading or reopening extent zero as long as the disk is not physically write protected. Error code 06 occurs whenever byte **R2** is nonzero under the current 2.0 release. Normally, nonzero return codes can be treated as missing data, with zero return codes indicating operation complete.

2.0.35. Write Random

Function 34: Write Random

Entry Parameters:	
Register C:	34 = 22H
Registers DE:	FCB Address
Returned Value:	
Register A:	Return code (see details)

The **Write Random** operation is initiated similarly to the Read Random call, except that data is written to the disk from the current DMA address. Further, if the disk extent or data block that is the target of the write has not yet been allocated, the allocation is performed before the write operation continues. As in the Read Random operation, the random record number is not changed as a result of the write. The logical extent number and current record positions of the FCB are set to correspond to the random record that is being written. Again, sequential read or write operations can begin following a random write, with the notation that the currently addressed record is either read or rewritten again as the sequential operation begins. You can also simply advance the random record position following each write to get the effect of a sequential write operation. Note that reading or writing the last record of an extent in random mode does not cause an automatic extent switch as it does in sequential mode.

The error codes returned by a random write are identical to the random read operation with the addition of error code 05, which indicates that a new extent cannot be created as a result of directory overflow.

2.0.36. Compute File Size

Function 35: Compute File Size

Entry Parameters:	
Register C:	35 = 23H
Registers DE:	FCB Address
Returned Value:	Random Record Field Set in FCB

When computing the size of a file, the DE register pair addresses an FCB in random mode format (bytes **R0**, **R1**, and **R2** are present). The FCB contains an unambiguous filename that is used in the directory scan. Upon return, the random record bytes contain the virtual file size, which is, in effect, the record address of the record following the end of the file. Following a call to Function

35, if the high record byte **R2** is 01, the file contains the maximum record count 65536. Otherwise, bytes **R0** and **R1** constitute a 16-bit value as before (**R0** is the least significant byte), which is the file size.

Data can be appended to the end of an existing file by simply calling Function 35 to set the random record position to the end of file and then performing a sequence of random writes starting at the preset record address.

The virtual size of a file corresponds to the physical size when the file is written sequentially. If the file was created in random mode and holes exist in the allocation, the file might contain fewer records than the size indicates. For example, if only the last record of an 8-megabyte file is written in random mode (that is, record number 65535), the virtual size is 65536 records, although only one block of data is actually allocated.

2.0.37. Set Random Record

Function 36: Set Random Record

Entry Parameters:	
Register C:	36 = 24H
Registers DE:	FCB Address
Returned Value:	Random Record Field Set in FCB

The Set Random Record function causes the BDOS automatically to produce the random record position from a file that has been read or written sequentially to a particular point. The function can be useful in two ways.

First, it is often necessary initially to read and scan a sequential file to extract the positions of various key fields. As each key is encountered, Function 36 is called to compute the random record position for the data corresponding to this key. If the data unit size is 128 bytes, the resulting record position is placed into a table with the key for later retrieval. After scanning the entire file and tabulating the keys and their record numbers, the user can move instantly to a particular keyed record by performing a random read, using the corresponding random record number that was saved earlier. The scheme is easily generalized for variable record lengths, because the program need only store the buffer-relative byte position along with the key and record number to find the exact starting position of the keyed data at a later time.

A second use of Function 36 occurs when switching from a sequential read or write over to random read or write. A file is sequentially accessed to a particular point in the file, Function 36 is called, which sets the record number, and subsequent random read and write operations continue from the selected point in the file.

2.0.38. Reset Drives

Function 37: Reset Drives

Entry Parameters:	
Register C:	37 = 25H
Registers DE:	Drive Vector
Returned Value:	
Register A:	00H

The Reset Drive function allows resetting of specified drives. The passed parameter is a 16-bit vector of drives to be reset; the least significant bit is drive A.

To maintain compatibility with MP/M, CP/M returns a zero value.

2.0.39. Access Drive

Function 38: Access Drive

Entry Parameters:	
Register C:	38 = 26H
Returned Value:	
Register A:	00H

This is an MP/M function that is not supported under CP/M. If called, the file system returns a zero in register A indicating that the access request is successful.

2.0.40. Free Drive

Function 39: Free Drive

Entry Parameters:	
Register C:	39 = 27H
Returned Value:	
Register A:	00H

This is an MP/M function that is not supported under CP/M. If called, the file system returns a zero in register A indicating that the free request is successful.

2.0.41. Write Random with Zero Fill

Function 40: Write Random with Zero Fill

Entry Parameters:	
Register C:	40 = 28H
Registers DE:	FCB Address
Returned Value:	
Register A:	Return code

The Write With Zero Fill operation is similar to Function 34, with the exception that a previously unallocated block is filled with zeros before the data is written.

3. A Sample File-to-File Copy Program

The following program provides a relatively simple example of file operations. The program source file is created as `COPY.ASM` using the CP/M ED program and then assembled using ASM or MAC, resulting in a HEX file. The LOAD program is used to produce a `COPY.COM` file that executes directly under the CCP. The program begins by setting the stack pointer to a local area and proceeds to move the second name from the default area at 006CH to a 33-byte File Control Block called DFCB. The DFCB is then prepared for file operations by clearing the current record field. At this point, the source and destination FCBs are ready for processing, because the SFCB at 005CH is properly set up by the CCP upon entry to the COPY program. That is, the first name is placed into the default FCB, with the proper fields zeroed, including the current record field at 007CH. The program continues by opening the source file, deleting any existing destination file, and creating the destination file. If all this is successful, the program loops at the label COPY until each record is read from the source file and placed into the destination file. Upon completion of the data transfer, the destination file is closed and the program returns to the CCP command level by jumping to BOOT.


```

;      sample file-to-file copy program
;
;      at the ccp level, the command
;
;      copy a:x.y b:u.v
;
;      copies the file named x.y from drive
;      a to a file named u.v. on drive b.
;
0000 = boot      equ 0000h      ;system reboot
0005 = bdos      equ 0005h      ;bdos entry point
005C = fcbl      equ 005ch      ;first file name
005C = sfcb      equ fcbl       ;source fcb
006C = fcb2      equ 006ch      ;second file name
0080 = dbuff     equ 0080h      ;default buffer
0100 = tpa       equ 0100h      ;beginning of tpa
;
0009 = printf    equ 9          ;print buffer func#
000F = openf     equ 15         ;open file func#
0010 = closef    equ 16         ;close file func#
0013 = deletef   equ 19         ;delete file func#
0014 = readf     equ 20         ;sequential read
0015 = writef    equ 21         ;sequential write
0016 = makef     equ 22         ;make file func#
;
0100          org tpa          ;beginning of tpa
0100 311A02    lxi sp,stack     ;local stack
;
;      move second file name to dfcb
0103 0E10      mvi c,16         ;half an fcb
0105 116C00    lxi d,fcbl       ;source of move
0108 21D901    lxi h,dfcb       ;destination fcb
010B 1A        mfcbl: ldax d     ;source fcb
010C 13        inx d            ;ready next
010D 77        mov m,a          ;dest fcb
010E 23        inx h            ;ready next
010F 0D        dcr c            ;count 16...0
0110 C20B01    jnz mfcbl       ;loop 16 times
;
;      name has been removed, zero cr
0113 AF        xra a            ;a = 00h
0114 32F901    sta dfcbl       ;current rec = 0
;
;      source and destination fcbs ready
;
0117 115C00    lxi d,sfcb       ;source file
011A CD6901    call open        ;error if 255
011D 118701    lxi d,nofile     ;ready message
0120 3C        inr a            ;255 becomes 0
0121 CC6101    cz finis         ;done if no file
;
;      source file open, prep destination
0124 11D901    lxi d,dfcb       ;destination
0127 CD7301    call delete      ;remove if present
;
012A 11D901    lxi d,dfcb       ;destination
012D CD8201    call make        ;create the file
0130 119601    lxi d,nodir      ;ready message
0133 3C        inr a            ;255 becomes 0
0134 CC6101    cz finis         ;done if no dir space
;
;      source file open, dest file open
;      copy until end of file on source
;
0137 115C00    copy: lxi d,sfcb  ;source
013A CD7801    call read        ;read next record
013D B7        ora a            ;end of file?
013E C25101    jnz eofile       ;skip write if so
;
;      not end of file, write the record
0141 11D901    lxi d,dfcb       ;destination
0144 CD7D01    call write       ;write record
0147 11A901    lxi d,space      ;ready message
014A B7        ora a            ;00 if write ok
014B C46101    cnz finis        ;end if so
014E C33701    jmp copy         ;loop until eof

```

```

;
; eofile: ;end of file, close destination
0151 11D901      lxi d,dfcb      ;destination
0154 CD6E01      call close      ;255 if error
0157 21BA01      lxi h,wrprot    ;ready message
015A 3C          inr a           ;255 becomes 00
015B CC6101      cz finis        ;should not happen
;
;
; copy operation complete, end
015E 11CB01      lxi d,normal    ;ready message
;
; finis      ;write message given by de, reboot
0161 0E09      mvi c,printf
0163 CD0500      call bdos        ;write message
0166 C30000      jmp boot         ;reboot system
;
;
; system interface subroutines
; (all return directly from bdos)
;
0169 0E0F      open: mvi c,openf
016B C30500      jmp bdos
;
016E 0E10      close: mvi c,closef
0170 C30500      jmp bdos
;
0173 0E13      delete mvi c,deletef
0175 C30500      jmp bdos
;
0178 0E14      read: mvi c,readf
017A C30500      jmp bdos
;
017D 0E15      write: mvi c,writef
017F C30500      jmp bdos
;
0182 0E16      make: mvi c,makef
0184 C30500      jmp bdos
;
;
; console messages
0187 6E6F20736Fnofile: db 'no source file$'
0196 6E6F206469nodir: db 'no directory space$'
01A9 6F7574206Fspace: db 'out of dat space$'
01BA 7772697465wrprot: db 'write protected?$'
01CB 636F707920normal: db 'copy complete$'
;
;
; data areas
01D9          ds 33          ;destination fcb
01F9 =        dfcbcr equ dfcb+32 ;current record
;
01FA          ds 32          ;16 level stack
;
stack:
021A          end

```

Note that there are several simplifications in this particular program. First, there are no checks for invalid filenames that could contain ambiguous references. This situation could be detected by scanning the 32-byte default area starting at location 005CH for ASCII question marks. A check should also be made to ensure that the filenames have been included (check locations 005DH and 006DH for non-blank ASCII characters). Finally, a check should be made to ensure that the source and destination filenames are different. An improvement in speed could be obtained by buffering more data on each read operation. One could, for example, determine the size of memory by fetching FBASE from location 0006H and using the entire remaining portion of memory for a data buffer. In this case, the programmer simply resets the DMA address to the next successive 128-byte area before each read. Upon writing to the destination file, the DMA address is reset to the beginning of the buffer and incremented by 128 bytes to the end as each record is transferred to the destination file.

4. A Sample File Dump Utility

The following file dump program is slightly more complex than the simple copy program given in the previous section. The dump program reads an input file, specified in the CCP command line, and displays the content of each record in hexadecimal format at the console. Note that the dump program saves the CCP's stack upon entry, resets the stack to a local area, and restores the CCP's stack before returning directly to the CCP. Thus, the dump program does not perform a warm start at the end of processing.

```

; FILE DUMP PROGRAM, READS AN INPUT FILE AND PRINTS IN HEX
;
; COPYRIGHT (C) 1975, 1976, 1977, 1978
; DIGITAL RESEARCH
; BOX 579, PACIFIC GROVE
; CALIFORNIA, 93950
;
0100      ORG      100H
0005 =    BDOS    EQU      0005H  ;DOS ENTRY POINT
0001 =    CONS    EQU      1      ;READ CONSOLE
0002 =    TYPEF   EQU      2      ;TYPE FUNCTION
0009 =    PRINTF  EQU      9      ;BUFFER PRINT ENTRY
000B =    BRKF    EQU      11     ;BREAK KEY FUNCTION (TRUE IF CHAR READY)
000F =    OPENF   EQU      15     ;FILE OPEN
0014 =    READF   EQU      20     ;READ FUNCTION
;
005C =    FCB     EQU      5CH    ;FILE CONTROL BLOCK ADDRESS
0080 =    BUFF    EQU      80H    ;INPUT DISK BUFFER ADDRESS
;
; NON GRAPHIC CHARACTERS
000D =    CR      EQU      0DH    ;CARRIAGE RETURN
000A =    LF      EQU      0AH    ;LINE FEED
;
; FILE CONTROL BLOCK DEFINITIONS
005C =    FCBDN   EQU      FCB+0  ;DISK NAME
005D =    FCBFN   EQU      FCB+1  ;FILE NAME
0065 =    FCBFT   EQU      FCB+9  ;DISK FILE TYPE (3 CHARACTERS)
0068 =    FCBRLL EQU      FCB+12  ;FILE'S CURRENT REEL NUMBER
006B =    FCBRCL EQU      FCB+15  ;FILE'S RECORD COUNT (0 TO 128)
007C =    FCBCR   EQU      FCB+32  ;CURRENT (NEXT) RECORD NUMBER (0 TO 127)
007D =    FCBLN   EQU      FCB+33  ;FCB LENGTH
;
; SET UP STACK
0100 210000    LXI      H,0
0103 39        DAD      SP
; ENTRY STACK POINTER IN HL FROM THE CCP
0104 221502    SHLD    OLDSP
; SET SP TO LOCAL STACK AREA (RESTORED AT FINIS)
0107 315702    LXI      SP,STKTOP
; READ AND PRINT SUCCESSIVE BUFFERS
010A CDC101    CALL     SETUP      ;SET UP INPUT FILE
010D FEFF      CPI      255       ;255 IF FILE NOT PRESENT
010F C21B01    JNZ      OPENOK    ;SKIP IF OPEN IS OK
;
; FILE NOT THERE, GIVE ERROR MESSAGE AND RETURN
0112 11F301    LXI      D,OPNMSG
0115 CD9C01    CALL     ERR
0118 C35101    JMP      FINIS     ;TO RETURN
;
OPENOK: ;OPEN OPERATION OK, SET BUFFER INDEX TO END
011B 3E80      MVI      A,80H
011D 321302    STA      IBP       ;SET BUFFER POINTER TO 80H
; HL CONTAINS NEXT ADDRESS TO PRINT
0120 210000    LXI      H,0       ;START WITH 0000
;
GLOOP:
0123 E5        PUSH     H         ;SAVE LINE POSITION
0124 CDA201    CALL     GNB        ;RECALL LINE POSITION
0127 E1        POP      H
0128 DA5101    JC       FINIS     ;CARRY SET BY GNB IF END FILE
012B 47        MOV      B,A
; PRINT HEX VALUES
; CHECK FOR LINE FOLD
012C 7D        MOV      A,L
012D E60F      ANI      0FH       ;CHECK LOW 4 BITS
012F C24401    JNZ      NONUM
; PRINT LINE NUMBER
0132 CD7201    CALL     CRLF
;
; CHECK FOR BREAK KEY
0135 CD5901    CALL     BREAK
; ACCUM LSB = 1 IF CHARACTER READY
0138 0F        RRC
0139 DA5101    JC       FINIS     ;DON'T PRINT ANY MORE
;
013C 7C        MOV      A,H
013D CD8F01    CALL     PHEX
0140 7D        MOV      A,L
0141 CD8F01    CALL     PHEX

```

```

NONUM:
0144 23      INX      H      ;TO NEXT LINE NUMBER
0145 3E20      MVI      A,' '
0147 CD6501     CALL     PCHAR
014A 78        MOV      A,B
014B CD8F01     CALL     PHEX
014E C32301     JMP      GLOOP
;
; FINIS:
;      END OF DUMP, RETURN TO CCP
;      (NOTE THAT A JMP TO 0000H REBOOTS)
0151 CD7201     CALL     CRLF
0154 2A1502     LHL      OLDSP
0157 F9        SPHL
;      STACK POINTER CONTAINS CCP'S STACK LOCATION
0158 C9        RET      ;TO THE CCP
;
;
; SUBROUTINES
;
BREAK: ;CHECK BREAK KEY (ACTUALLY ANY KEY WILL DO)
0159 E5D5C5     PUSH    H! PUSH D! PUSH B; ENVIRONMENT SAVED
015C 0E0B      MVI      C,BRKF
015E CD0500     CALL     BDOS
0161 C1D1E1     POP     B! POP D! POP H; ENVIRONMENT RESTORED
0164 C9        RET
;
PCHAR: ;PRINT A CHARACTER
0165 E5D5C5     PUSH    H! PUSH D! PUSH B; SAVED
0168 0E02      MVI      C,TYPEF
016A 5F        MOV      E,A
016B CD0500     CALL     BDOS
016E C1D1E1     POP     B! POP D! POP H; RESTORED
0171 C9        RET
;
CRLF:
0172 3E0D      MVI      A,CR
0174 CD6501     CALL     PCHAR
0177 3E0A      MVI      A,LF
0179 CD6501     CALL     PCHAR
017C C9        RET
;
;
; PNIB: ;PRINT NIBBLE IN REG A
017D E60F      ANI      0FH      ;LOW 4 BITS
017F FE0A      CPI      10
0181 D28901     JNC      P10
;      LESS THAN OR EQUAL TO 9
0184 C630      ADI      '0'
0186 C38B01     JMP      PRN
;
;      GREATER OR EQUAL TO 10
0189 C637      P10:     ADI      'A' - 10
018B CD6501     PRN:     CALL     PCHAR
018E C9        RET
;
PHEX: ;PRINT HEX CHAR IN REG A
018F F5        PUSH     PSW
0190 0F        RRC
0191 0F        RRC
0192 0F        RRC
0193 0F        RRC
0194 CD7D01     CALL     PNIB      ;PRINT NIBBLE
0197 F1        POP      PSW
0198 CD7D01     CALL     PNIB
019B C9        RET
;
ERR: ;PRINT ERROR MESSAGE
;      D,E ADDRESSES MESSAGE ENDING WITH "$"
019C 0E09      MVI      C,PRINTF      ;PRINT BUFFER FUNCTION
019E CD0500     CALL     BDOS
01A1 C9        RET

```

```

;
;
GNB: ;GET NEXT BYTE
01A2 3A1302 LDA IBP
01A5 FE80 CPI 80H
01A7 C2B301 JNZ G0
; READ ANOTHER BUFFER
;
;
01AA CDCE01 CALL DISKR
01AD B7 ORA A ;ZERO VALUE IF READ OK
01AE CAB301 JZ G0 ;FOR ANOTHER BYTE
; END OF DATA, RETURN WITH CARRY SET FOR EOF
01B1 37 STC
01B2 C9 RET
;
G0: ;READ THE BYTE AT BUFF+REG A
01B3 5F MOV E,A ;LS BYTE OF BUFFER INDEX
01B4 1600 MVI D,0 ;DOUBLE PRECISION INDEX TO DE
01B6 3C INR A ;INDEX=INDEX+1
01B7 321302 STA IBP ;BACK TO MEMORY
; POINTER IS INCREMENTED
; SAVE THE CURRENT FILE ADDRESS
01BA 218000 LXI H,BUFF
01BD 19 DAD D
; ABSOLUTE CHARACTER ADDRESS IS IN HL
01BE 7E MOV A,M
; BYTE IS IN THE ACCUMULATOR
01BF B7 ORA A ;RESET CARRY BIT
01C0 C9 RET
;
SETUP: ;SET UP FILE
; OPEN THE FILE FOR INPUT
01C1 AF XRA A ;ZERO TO ACCUM
01C2 327C00 STA FCBCR ;CLEAR CURRENT RECORD
;
01C5 115C00 LXI D,FCB
01C8 0E0F MVI C,OPENF
01CA CD0500 CALL BDOS
; 255 IN ACCUM IF OPEN ERROR
01CD C9 RET
;
DISKR: ;READ DISK FILE RECORD
01CE E5D5C5 PUSH H! PUSH D! PUSH B
01D1 115C00 LXI D,FCB
01D4 0E14 MVI C,READF
01D6 CD0500 CALL BDOS
01D9 C1D1E1 POP B! POP D! POP H
01DC C9 RET
;
; FIXED MESSAGE AREA
01DD 46494C4520SIGNON: DB 'FILE DUMP VERSION 1.4$'
01F3 0D0A4E4F20OPNMSG: DB CR,LF,'NO INPUT FILE PRESENT ON DISK$'
;
; VARIABLE AREA
0213 IBP: DS 2 ;INPUT BUFFER POINTER
0215 OLDSP: DS 2 ;ENTRY SP VALUE FROM CCP
;
; STACK AREA
0217 DS 64 ;RESERVE 32 LEVEL STACK
STKTOP:
;
0257 END

```

5. A Sample Random Access Program

This chapter concludes with an extensive example of random access operation. The program listed below performs the simple function of reading or writing random records upon command from the terminal. When a program has been created, assembled, and placed into a file labeled RANDOM.COM, the CCP level command

RANDOM X.DAT

starts the test program. The program looks for a file by the name X.DAT and, if found, proceeds to prompt the console for input. If not found, the file is created before the prompt is given. Each prompt takes the form

next command?

and is followed by operator input, followed by a carriage return. The input commands take the form

nW nR Q

where *n* is an integer value in the range 0 to 65535, and W, R, and Q are simple command characters corresponding to random write, random read, and quit processing, respectively. If the W command is issued, the RANDOM program issues the prompt

type data:

The operator then responds by typing up to 127 characters, followed by a carriage return. RANDOM then writes the character string into the X.DAT file at record *n*. If the R command is issued, RANDOM reads record number *n* and displays the string value at the console. If the Q command is issued, the X.DAT file is closed, and the program returns to the CCP. In the interest of brevity, the only error message is

error, try again.

The program begins with an initialization section where the input file is opened or created, followed by a continuous loop at the label ready where the individual commands are interpreted. The DFBC at 005CH and the default buffer at 0080H are used in all disk operations. The utility subroutines then follow, which contain the principal input line processor, called readc. This particular program shows the elements of random access processing, and can be used as the basis for further program development.

Sample Random Access Program for CP/M 2.0

```

0100          org      100h      ;base of tpa

;
0000 =        reboot equ     0000h      ;system reboot
0005 =        bdos   equ     0005h      ;bdos entry point
;
0001 =        coninp equ     1          ;console input function
0002 =        conout equ     2          ;console output function
0009 =        pstring equ     9          ;print string until '$'
000A =        rstring equ     10         ;read console buffer
000C =        version equ     12         ;return version number
000F =        openf  equ     15         ;file open function
0010 =        closef equ     16         ;close function
0016 =        makef  equ     22         ;make file function
0021 =        readr  equ     33         ;read random
0022 =        writer equ     34         ;write random
;
005C =        fcb    equ     005ch      ;default file control
;block
007D =        ranrec equ     fcb+33      ;random record position
007F =        ranovf equ     fcb+35      ;high order (overflow)
;byte
0080 =        buff   equ     0080h      ;buffer address
;
000D =        cr     equ     0dh        ;carriage return
000A =        lf     equ     0ah        ;line feed
;
;      Load SP, Set-Up File for Random Access
;
0100 31BC02          lxi     sp,stack
;
;      version 2.0
0103 0E0C          mvi     c,version
0105 CD0500          call    bdos
0108 FE20          cpi     20h          ;version 2.0 or better?
010A D21601          jnc     versok
;      bad version, message and go back
010D 111B02          lxi     d,badver
0110 CDDA01          call    print
0113 C30000          jmp     reboot
;
versok:
;      correct versionm for random access
0116 0E0F          mvi     c,openf      ;open default fcb
0118 115C00          lxi     d,fcb
011B CD0500          call    bdos
011E 3C          inr     a              ;err 255 becomes zero
011F C23701          jnz     ready
;
;      cannot open file, so create it
0122 0E16          mvi     c,makef
0124 115C00          lxi     d,fcb
0127 CD0500          call    bdos
012A 3C          inr     a              ;err 255 becomes zero
012B C23701          jnz     ready
;
;      cannot create file, directory full
012E 113A02          lxi     d,nospace
0131 CDDA01          call    print
0134 C30000          jmp     reboot      ;back to ccp
;
;      Loop Back to Ready After Each Command
;
ready:
;      file is ready for processing
;
0137 CDE501          call    readcom      ;read next command
013A 227D00          shld    ranrec      ;store input record#
013D 217F00          lxi     h,ranovf
0140 3600          mvi     m,0          ;clear high byte if set
0142 FE51          cpi     'Q'          ;quit?
0144 C25601          jnz     notq
;
;      quit processing, close file
0147 0E10          mvi     c,closef
0149 115C00          lxi     d,fcb
014C CD0500          call    bdos
014F 3C          inr     a              ;err 255 becomes 0
0150 CAB901          jz     error        ;error message, retry
0153 C30000          jmp     reboot      ;back to ccp
;
;
;      End of Quit Command, Process Write

```



```

;
notq:
; not the quit command, random write?
0156 FE57 cpi 'W'
0158 C28901 jnz notw

;
; this is a random write, fill buffer untill cr
015B 114D02 lxi d,datmsg
015E CDDA01 call print ;data prompt
0161 0E7F mvi c,127 ;up to 127 characters
0163 218000 lxi h,buff ;destination
rloop: ;read next character to buff
0166 C5 push b ;save counter
0167 E5 push h ;next destination
0168 CDC201 call getchr ;character to a
016B E1 pop h ;restore counter
016C C1 pop b ;restore next to fill
016D FE0D cpi cr ;end of line?
016F CA7801 jz erloop
; not end, store character
0172 77 mov m,a
0173 23 inx h ;next to fill
0174 0D dcr c ;counter goes down
0175 C26601 jnz rloop ;end of buffer?
erloop:
; end of read loop, store 00
0178 3600 mvi m,0

;
; write the record to selected record number
017A 0E22 mvi c,writer
017C 115C00 lxi d,fcf
017F CD0500 call bdos
0182 B7 ora a ;erro code zero?
0183 C2B901 jnz error ;message if not
0186 C33701 jmp ready ;for another record

;
; End of Write Command, Process Read
;
notw:
; not a write command, read record?
0189 FE52 cpi 'R'
018B C2B901 jnz error ;skip if not

;
; read random record
018E 0E21 mvi c,readr
0190 115C00 lxi d,fcf
0193 CD0500 call bdos
0196 B7 ora a ;return code 00?
0197 C2B901 jnz error

;
; read was successful, write to console
019A CDCF01 call crlf ;new line
019D 0E80 mvi c,128 ;max 128 characters
019F 218000 lxi h,buff ;next to get
wloop:
01A2 7E mov a,m ;next character
01A3 23 inx h ;next to get
01A4 E67F ani 7fh ;mask parity
01A6 CA3701 jz ready ;for another command
; if 00
01A9 C5 push b ;save counter
01AA E5 push h ;save next to get
01AB FE20 cpi ' ' ;graphic?
01AD D4C801 cnc putchr ;skip output if not
01B0 E1 pop h
01B1 C1 pop b
01B2 0D dcr c ;count=count-1
01B3 C2A201 jnz wloop
01B6 C33701 jmp ready

;
; End of Read Command, All Errors End Up Here
;
error:
01B9 115902 lxi d,errmsg
01BC CDDA01 call print
01BF C33701 jmp ready
;

```

```

getchr:
;read next console character to a
01C2 0E01      mvi    c,coninp
01C4 CD0500    call   bdos
01C7 C9        ret

;
putchr:
;write character from a to console
01C8 0E02      mvi    c,conout
01CA 5F        mov     e,a      ;character to send
01CB CD0500    call   bdos      ;send character
01CE C9        ret

;
crlf:
;send carriage return line feed
01CF 3E0D      mvi    a,cr      ;carriage return
01D1 CDC801    call   putchr
01D4 3E0A      mvi    a,lf      ;line feed
01D6 CDC801    call   putchr
01D9 C9        ret

;
print:
;print the buffer addressed by de untill $
01DA D5        push   d
01DB CDCF01    call   crlf
01DE D1        pop     d          ;new line
01DF 0E09      mvi    c,pstring
01E1 CD0500    call   bdos      ;print the string
01E4 C9        ret

;
readcom:
;read the next command line to the conbuf
01E5 116B02    lxi    d,prompt
01E8 CDDA01    call   print      ;command?
01EB 0E0A      mvi    c,rstring
01ED 117A02    lxi    d,conbuf
01F0 CD0500    call   bdos      ;read command line
;
;command line is present, scan it
01F3 210000    lxi    h,0      ;start with 0000
01F6 117C02    lxi    d,conlin  ;command line
01F9 1A        readc: ldax   d      ;next command
;character
01FA 13        inx     d      ;to next command
;position
01FB B7        ora     a      ;cannot be end of
;command
01FC C8        rz
;not zero, numeric?
01FD D630      sui     '0'
01FF FE0A      cpi     10      ;carry if numeric
0201 D21302    jnc     endrd
;
;add-in next digit
0204 29        dad     h      ;*2
0205 4D        mov     c,l
0206 44        mov     b,h      ;bc = value * 2
0207 29        dad     h      ;*4
0208 29        dad     h      ;*8
0209 09        dad     b      ;*2 + *8 = *10
020A 85        add     l      ;*digit
020B 6F        mov     l,a
020C D2F901    jnc     readc    ;for another char
020F 24        inr     h      ;overflow
0210 C3F901    jmp     readc    ;for another char
endrd:
;
;end of read, restore value in a
0213 C630      adi     '0'      ;command
0215 FE61      cpi     'a'      ;translate case?
0217 D8        rc
;
;lower case, mask lower case bits
0218 E65F      ani     101$1111b
021A C9        ret

```

```

;
;      String Data Area for Console Messages
;
badver:
021B 736F727279      db      'sorry, you need cp/m version 2$'
nospace:
023A 6E6F206469      db      'no directory space$'
datmsg:
024D 7479706520      db      'type data: $'
errmsg:
0259 6572726F72      db      'error, try again.$'
prompt:
026B 6E65787420      db      'next command? $'
;
;      Fixed and Variable Data Area
;
;
027A 21      conbuf: db      conlen      ;length of console buffer
027B      consiz: ds      1      ;resulting size after read
027C      conlin: ds      32      ;length 32 buffer
0021 =      conlen equ      $-consiz
;
029C      ds      32      ;16 level stack
stack:
02BC      end

```

Major improvements could be made to this particular program to enhance its operation. In fact, with some work, this program could evolve into a simple data base management system. One could, for example, assume a standard record size of 128 bytes, consisting to arbitrary fields within the record. A program, called GETKEY, could be developed that first reads a sequential file and extracts a specific field defined by the operator. For example, the command

```
GETKEY NAMES.DAT LASTNAME 10 20
```

would cause GETKEY to read the data base file NAMES.DAT and extract the LASTNAME field from each record, starting in position 10 and ending at character 20. GETKEY builds a table in memory consisting of each particular LASTNAME field, along with its 16-bit record number location within the file. The GETKEY program then sorts this list and writes a new file, called LASTNAME.KEY, which is an alphabetical list of LASTNAME fields with their corresponding record numbers. This list is called an inverted index in information retrieval parlance.

If the programmer were to rename the program shown above as QUERY and modify it so that it reads a sorted key file into memory, the command line might appear as

```
QUERY NAMES.DAT LASTNAME.KEY
```

Instead of reading a number, the QUERY program reads an alphanumeric string that is a particular key to find in the NAMES.DAT data base. Because the LASTNAME.KEY list is sorted, one can find a particular entry rapidly by performing a binary search, similar to looking up a name in the telephone book. Starting at both ends of the list, one examines the entry halfway in between and, if not matched, splits either the upper half or the lower half for the next search. You will quickly reach the item you are looking for and find the corresponding record number. You should fetch and display this record at the console, just as was done in the program shown above.

With some more work, you can allow a fixed grouping size that differs from the 128-byte record shown above. This is accomplished by keeping track of the record number and the byte offset within the record. Knowing the group size, you randomly access the record containing the proper group, offset to the beginning of the group within the record read sequentially until the group size has been exhausted.

Finally, you can improve QUERY considerably by allowing boolean expressions, which compute the set of records that satisfy several relationships, such as a LASTNAME between HARDY and LAUREL and an AGE lower than 45. Display all the records that fit this description. Finally, if your lists are getting too big to fit into memory, randomly access key files from the disk as well.

6. System Function Summary

#	Name	Parameters	Results
0	System Reset	none	Does not return
1	Console Input	none	A: ASCII Character
2	Console Output	E: ASCII character to output	none
3	Reader Input	none	A: ASCII Character
4	Punch Output	E: ASCII Character	none
5	List Output	E: ASCII Character	none
6	Direct Console I/O	E: 0FFH (input) or char (output)	A: character or status
7	Get IOBYTE	none	A: IOBYTE value
8	Set IOBYTE	E: IOBYTE value	none
9	Print String	D E: String Address	none
10	Read Console Buffer	D E: Buffer Address	Characters input are in the Buffer
11	Get Console Status	none	A: 0FFH if a character is ready, 0 if not
12	Version Number	none	HL: Version Number
13	Reset Disk System	none	A: 0FFH if \$ file present, 0 if not
14	Select Disk	E: Drive number: 0 for A, 1 for B, ...	A: 0 if successful, 0FFH on error
15	Open File	D E: FCB Address	A: Directory Code
16	Close File	D E: FCB Address	A: Directory Code
17	Search for First	D E: FCB Address	A: Directory Code
18	Search for Next	none	A: Directory Code
19	Delete File	D E: FCB Address	A: Directory Code
20	Read Sequential	D E: FCB Address	A: Directory Code
21	Write Sequential	D E: FCB Address	A: Directory Code

#	Name	Parameters	Results
22	Make File	DE: FCB Address	A: Directory Code
23	Rename File	DE: FCB Address	A: Directory Code
24	Return Login Vector	none	HL: Log-in Vector
25	Return Current Disk	none	A: Current Disk. 0 for A, ... 15 for P
26	Set DMA Address	DE: DMA Address	none
27	Get Addr(ALLOC)	none	HL: ALLOC Address
28	Write Protect Disk	none	none
29	Get R/O Vector	none	HL: R/O Vector Value
30	Set File Attributes	DE: FCB Address	A: Directory Code
31	Get Addr(Disk Parameter Block)	none	HL: DPB Address
32	Set/Get User Code	E: 0FFH (get) or <i>User Code</i> (set)	A: Current code (get) or none (set)
33	Read Random	DE: FCB Address	A: Return code (see details)
34	Write Random	DE: FCB Address	A: Return code (see details)
35	Compute File Size	DE: FCB Address	Random Record Field Set in FCB
36	Set Random Record	DE: FCB Address	Random Record Field Set in FCB

BDOS System Function Summary

Notes on the Zcim Project Edition

The *Zcim Project* goal is to improve the accessibility of legacy Z80 CP/M era content. As part of the project, Jim Burlingame (jb@samplx.org) created a *Typst* version of this manual.

You can find out more about the project at its site z80cim.org. The sources of this document are available on GitHub.

The source material is from the *Tim Olmstead Memorial Digital Research CP/M Library*

The individual documents from the archive include:

- *CP/M 2.0 Interface Guide* (PDF)

The contents of the manual were edited using the *Typst.app* site.

The sources of the document are available at GitHub.

License

The source documentation is under a license granted by the owner of the Digital Research intellectual property in an email available at <http://cpm.z80.de/license.html>.

The *Zcim Project* edition is under the Creative Commons Attribution 4.0 International license.

CC BY 4.0.